

Group 9 Members:

- Laurenz Luke Gonzales
- Mahooba Hashimi
- Saviour Msopa

GitHub Link: https://github.com/lgonza32/CSCI6461_Group9_Project.git
Guide/Instructions show in GitHub [Readme.md](#)

Additional Notes and Test Cases found in: /docs/part3_test_cases/

Goal for Part 3:

- Finish the remaining required instructions for this part, especially CHK and TRAP.
- Run Program 2 on the simulator from a machine-code load file.
- Make sure the simulator can search a paragraph for a word and print the correct result.

Main Design:

The overall simulator design stayed the same as Part II:

- Assembler builds listing and load files
- CPU / Memory / MachineState run instructions and hold machine state
- GUI / Controller handles IPL, run, single step, input, and printer output
- Tests / Programs verify both single instructions and full program execution

CHK and TRAP Design:

- CHK and TRAP were grouped together because both were remaining instructions that needed to be completed in Part III.
- CHK
 - Implemented as a device-status instruction
 - Uses the same device setup already used for IN and OUT
 - Returns device status into a register
- TRAP
 - Implemented using the ISA trap rules
 - Saves PC + 1 in reserved memory
 - Uses memory location 0 as the trap table base
 - Uses the trap code as an index into the table
 - Transfers control to the selected trap routine

Program 2 Design:

- The final version is organized into code pages and data pages:
 - boot page
 - main page
 - print pages
 - input page
 - search pages
 - compare pages
 - result pages

- paragraph page
- work page
- Main tasks:
 - print a paragraph of 6 sentences
 - ask the user for a word
 - search the paragraph
 - if found, print:
 - FOUND
 - the word
 - the sentence number
 - the word number
 - otherwise print NOT FOUND
- Matching is whole-word exact match, not substring matching

Controller and GUI Design

- The GUI and CPU logic stayed separate
- The controller still handles:
 - IPL
 - Run
 - single step
 - Halt
 - Reset
 - manual memory/register actions
 - console input
 - printer output
- Program 2 mainly uses:
 - Console Input for the typed word
 - Printer Output for paragraph and search results

Instruction Testing:

- InstructionTests.java: Main way to test instructions
 - Instruction-level testing was expanded for:
 - CHK
 - TRAP
- Program-level testing used Program2.asm to verify
 - file loading
 - paragraph printing
 - typed-word input
 - search correctness
 - FOUND / NOT FOUND behavior
 - Bounds logic